Neural Rover: Design and Implementation of a Wi-Fi-Controlled Mobile Robot with Live Pan-Tilt Video

Jubayed Ahmed Sahel, Fahad Hossain Shourov, Pujon Chakraborty, and Anonna Rahman Spondon
Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh
Supervisor: Dr. K. M. A. Salam

***Abstract*-**This paper presents Neural Rover, a four-wheel mobile robot controlled from a browser over a 2.4-GHz Wi-Fi network. A 30-pin ESP32 hosts a responsive WebSocket interface, drives four DC geared motors through two L298N dual H-bridges, controls a two-axis camera mount, and enforces fail-safe stopping. A separate ESP32-CAM supplies live video to the same page, while a 3S battery and LM2596 provide separated motor and regulated 5-V rails. Validation covered motion, steering, held commands, stopping, camera discovery, live video, pan-tilt operation, and reconnection. All eight planned CSE499A milestones were implemented and verified, giving 8/8 (100%) workplan completion. Face recognition remains a later laptop-side extension.

***Index Terms*-**ESP32, ESP32-CAM, mobile robot, WebSocket, L298N, pan-tilt camera, remote control

I. Introduction

Mobile robots used for inspection, teaching, and response experiments require both controllable motion and dependable visual feedback. Wireless reliability matters because degraded communication directly affects the operator's ability to command and observe the platform. Mobile-robot research has demonstrated that a phone can combine Wi-Fi teleoperation and visual functions in one operator interface [7]. Neural Rover addresses the same control-and-observation problem with a compact local system that does not depend on an Internet cloud service.

The ESP32 was selected as the primary controller because one device combines 2.4-GHz Wi-Fi, programmable GPIO, multiple UART interfaces, and PWM peripherals [1]. A second ESP32 camera board separates image capture and streaming from time-critical motor control. Espressif's camera driver supports common sensors, frame-buffer management, and format-conversion paths required for browser streaming [2]. This two-board organization keeps the motor safety loop available while the camera captures or converts a frame.

The CSE499A objective was to build and verify a rover that can be driven from a phone or laptop, display live video, aim the camera using pan and tilt, and enter a safe stopped state whenever control messages disappear. The eight-week workplan defined eight milestones; all eight are complete and verified, so 100% of the 499A workplan is finished. Face recognition is excluded from this completion count and remains future laptop-side work. Project records are maintained in the public repository [6].

The eight-week workplan covered component selection, chassis assembly, motor wiring, browser control, camera streaming, pan-tilt integration, communication reliability, complete-system testing, demonstration, and reporting. Every required CSE499A deliverable has been implemented and checked [6]. This report documents the completed embedded platform; recognition algorithms are discussed only as possible later work.

II. Purpose and Engineering Requirements

The main objective was intuitive teleoperation with useful visual awareness using inexpensive, accessible modules. The design therefore had to provide directional mobility, prompt response to a held control, video on the same interface, independent camera aiming, understandable network diagnostics, and a safe response to communication failure. WebSocket was selected because it maintains a persistent full-duplex TCP connection instead of requiring a new HTTP request for every repeated command [5].

Table I translates the purpose into acceptance-oriented engineering requirements. The criteria distinguish a feature appearing on a webpage from the complete electromechanical behavior required on the physical vehicle. Forward is accepted only when all four wheels agree, while fail-safe behavior is accepted only when every enable output returns to zero after release, timeout, or loss of connection [6].

The completed baseline is suitable for indoor visual inspection, laboratory teleoperation, embedded-systems education, line-following experiments, and later object-detection studies. These applications depend on the same verified combination of motion, video feedback, local networking, camera aiming, and safe stopping.

III. Methodology and Implementation

A. System Architecture

Fig. 1 shows the implemented responsibility and power boundaries. The phone or laptop and both ESP32 boards join the same 2.4-GHz `CAR` network. The browser opens the main controller on port 80 and maintains `/ws` for motion and servo messages. The ESP32-CAM serves a diagnostic page on port 80 and its MJPEG stream on port 81. UART2 lets the camera

TABLE I
DESIGN REQUIREMENTS AND ACCEPTANCE CRITERIA

Requirement	Acceptance criterion
Mobility	Four wheels produce correct forward, backward, left, and right motion.
Responsiveness	A held control maintains motion without page navigation or repeated reconnects.
Visual feedback	Live camera frames appear inside the controller page.
View control	Pan and tilt respond independently across a usable mechanical range.
Safety	Release, stale command, browser loss, or Wi-Fi loss stops every motor channel.
Diagnostics	Numeric and fixed local endpoints are reported for controller and camera.
Power integrity	Logic remains operational while motors and servos are commanded.

board report its assigned address to the main controller, which then supplies the correct stream host to the webpage [6].

The main ESP32 owns every actuator and the stop decision. The ESP32-CAM owns capture, conversion, stream delivery, and illumination. The browser is a client rather than a safety controller: if it disappears, the rover stops locally. A stalled stream or closed page therefore cannot directly block the motor-control loop.

B. Motor, Servo, and UART Interfaces

Two L298N modules provide four H-bridge channels. The L298 is a dual full-bridge driver with separate enable inputs, making each board suitable for two reversible DC motors [3]. ENA and ENB jumpers were removed so the main ESP32 can apply PWM. Table II gives the authoritative final physical assignment. It supersedes preliminary side labels: the GPIO32 group is the physical left driver and the GPIO25 group is the physical right driver. The front-left polarity is corrected in software so that all four wheels agree with Forward [6].

The ESP32 LEDC peripheral supports independent PWM waveforms with configurable frequency and duty resolution [1]. Four 12-kHz, 8-bit outputs drive the motor enable pins, while two 50-Hz, 16-bit outputs generate servo pulses. Pan uses GPIO13 and tilt uses GPIO4 [6].

UART2 runs at 115200 baud. Main RX2 (GPIO16) receives ESP32-CAM U0T, main TX2 (GPIO17) connects to ESP32-CAM U0R, and ground is common. The camera repeatedly sends a short `CAMIP:` message; line-oriented parsing validates and stores the address without carrying the video stream over UART [6].

C. Power Distribution and Electrical Integrity

The 3S battery supplies both L298N motor inputs and the LM2596 input. The LM2596 is a 150-kHz step-down regulator family rated for loads up to 3 A when correctly implemented and thermally managed [4]. Its output is adjusted to 5.0 V before connection and supplies the main ESP32, ESP32-CAM, and two servos. Battery negative, regulator negative, both controller grounds, both driver grounds, and servo grounds are joined as one signal reference [6].

TABLE II
FINAL MAIN-ESP32 PIN ASSIGNMENT

Subsystem	Signal	GPIO
Left L298N	ENA, IN1, IN2	32, 23, 22
	IN3, IN4, ENB	21, 19, 18
Right L298N	ENA, IN1, IN2	25, 26, 27
	IN3, IN4, ENB	14, 12, 33
Pan-tilt	Pan, tilt	13, 4
UART2	RX2, TX2	16, 17

Power debugging was treated as an integration task rather than assuming successful USB tests proved vehicle operation. Motor starting current and switching noise can produce resets, delayed responses, or corrupted serial characters when the logic supply or ground is disturbed. The final arrangement separates the battery motor rail from the regulated logic rail, avoids competing USB and buck-converter 5-V sources during normal operation, and uses short, secure power conductors.

The L298N enable jumpers and its 5V-EN jumper have different purposes. ENA and ENB jumpers remain removed because the ESP32 supplies PWM. The 5V-EN jumper is configured according to whether the module uses its onboard regulator or an external 5-V logic rail. Treating these jumpers separately prevents accidental connection of competing supplies [3], [6].

D. Control Firmware and Safety State Machine

The main firmware joins `CAR`, starts an asynchronous web server, and exposes `/ws`. The browser sends F, B, L, or R while a button is held, S on release, and periodic K keepalives. It also sends P0–P180 and T0–T180 servo targets. This persistent, full-duplex model follows RFC 6455 and prevents movement buttons from navigating away from the page [5], [6].

Each motion request passes through a local state machine. PWM begins at duty 120 and rises by 12 every 12 ms toward 255. If direction changes, duty is set to zero, all direction inputs are neutralized, a 40-ms dead time is observed, and only then is the new polarity applied. If no valid movement or keepalive arrives within 1300 ms, or WebSocket/Wi-Fi disconnects, every enable output returns to zero. Safety therefore does not depend on the browser successfully sending a final Stop packet [6].

The ramp overcomes motor starting friction without requesting the full four-motor load in one software step and makes direction changes less abrupt. It is not closed-loop speed control because no wheel encoders are installed; actual speed can vary with battery voltage, surface load, and motor differences.

E. Camera and Network Integration

The camera firmware first attempts VGA hardware-JPEG capture. The installed sensor reports that JPEG is unsupported, so initialization is repeated with RGB565 at QVGA (320×240), one frame buffer, and software JPEG conversion. Espressif notes that RGB/YUV capture creates greater

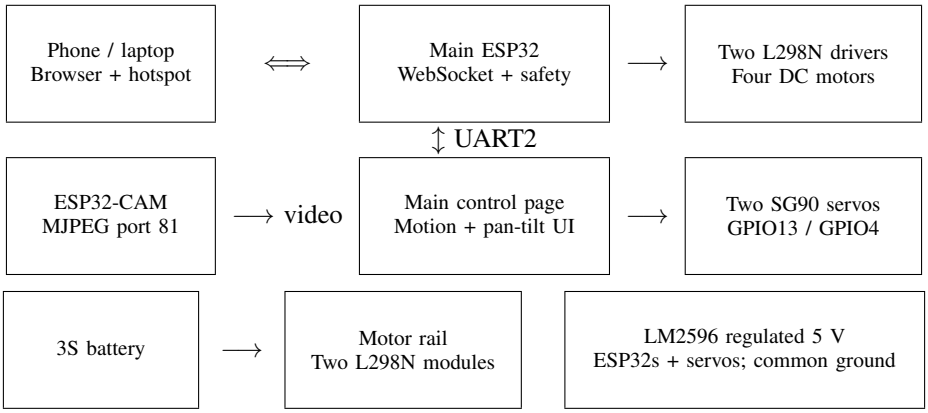


Fig. 1. Implemented control, video, actuation, and power boundaries. The final GPIO mapping is given in Table II.

processor and memory load and recommends lower resolutions when JPEG cannot be produced by the sensor [2]. The fallback prioritizes stable streaming over resolution.

Both boards operate as stations on the same hotspot. The main page is available through `rover.local` and the camera through `rovercam.local`; numeric DHCP addresses printed in Serial Monitor remain the fallback. Reconnection logic attempts to restore Wi-Fi without retaining movement. The interface exposes connection state and converts loss into a stop instead of hiding it [6].

F. Staged Implementation Procedure

Integration proceeded from low-risk subsystems to the complete vehicle. First, chassis motor pairs and driver outputs were identified without relying on assumed side labels. Second, each direction channel was tested with wheels raised and polarity was recorded. Third, battery and regulated rails were checked separately. Fourth, the main ESP32 served a minimal movement page before the camera load was introduced. Fifth, the camera stream and UART discovery were verified independently. Finally, servos, video, WebSocket control, motor ramping, and fail-safe behavior were exercised together [6].

This order distinguished software faults from power faults. A motor that moved in an isolated test demonstrated a functioning output channel, while resets after connecting a particular channel identified wiring, driver, motor, or noise problems. Conversely, correct command logs with no vehicle movement directed attention to enable wiring and power rather than the webpage.

IV. Results, Discussion, and Analysis

A. CSE499A Workplan Completion

The eight-week CSE499A plan contained eight milestones covering components and circuit planning, chassis and motor wiring, phone control, camera streaming, pan-tilt integration, communication and safety, integrated testing, and the final demonstration/report. Each milestone has verified implementation evidence. Therefore, 8 of 8 items are complete, corresponding to 100% workplan completion [6].

B. Functional Validation

Validation used the final wiring and firmware. Movement testing began from Stop with wheels raised and was repeated on the floor after direction agreement was confirmed. A held-command test checked that motion persisted while the pointer remained pressed and stopped on release. Loss tests closed the browser connection or interrupted message delivery and verified that the watchdog returned the enables to zero. Camera tests checked both the independent camera page and the stream embedded in the controller [6].

Table III summarizes the acceptance observations. The tests are functional rather than instrumented: they establish correct integrated behavior but do not provide measured latency, radio range, current, or frame rate. A later quantitative study should add repeatable command-to-actuation timing, packet-loss, range, current, and stream-rate measurements.

C. Fault Isolation and Corrections

The final result depended on separating browser, power, and wiring symptoms. Table IV records the most important corrections so that maintenance does not repeat the same investigation.

D. Motion and User-Interface Analysis

The decisive motor correction was an accurate mapping between physical sides, driver channels, and software names. Preliminary labels assigned the driver groups oppositely, and the front-left motor required reversed logical polarity. Correcting these relationships in one software layer produced consistent forward rotation while leaving already-correct motors unchanged. The final terminal arrangement is now documented to prevent regression [6].

The WebSocket interface improved usability because held events update the controller over an existing connection. Release sends Stop immediately, while the watchdog remains an independent backup. The page prevents touch interaction from creating browser navigation. This matches the persistent bidirectional purpose of WebSocket while keeping the safety action inside the ESP32 [5].

TABLE III
INTEGRATED FUNCTIONAL TEST RESULTS

Test	Acceptance observation	Result
Forward/backward	Four wheels agree with command	Pass
Left/right turn	Correct differential direction	Pass
Held command	Motion continues while held	Pass
Release/timeout	Outputs return to stopped state	Pass
Live video	Stream visible in control page	Pass
Camera discovery	UART address reaches main page	Pass
Pan and tilt	Both axes respond to controls	Pass
Reconnect	Control resumes after reconnection	Pass

TABLE IV
OBSERVED FAULTS AND IMPLEMENTED CORRECTIONS

Observation	Cause identified	Correction
Only initial command responded	Request/page control and unstable power experiments	Persistent WebSocket events plus local timeout
ESP32 reset under motor load	Logic-rail disturbance and motor transients	Separate motor/regulated rails; common ground
Front wheels opposed Forward	Preliminary side labels and one polarity mismatch	Final physical mapping and software inversion
Camera address changed	DHCP reassignment after hotspot reconnect	mDNS, serial endpoints, and UART discovery
JPEG initialization failed	Sensor lacks hardware JPEG support	RGB565 QVGA with software conversion

The final baseline does not expose a speed slider. Earlier experiments added interface complexity without resolving power and connection faults. The tested version ramps automatically to full duty and prioritizes predictable direction, stopping, and connection behavior. Future speed control should follow encoder feedback or repeatable duty-versus-speed characterization.

E. Video, Communication, and Power Analysis

Separating camera and motor workloads proved useful. The main controller remains responsible for movement and safety while the ESP32-CAM performs capture and software JPEG conversion. QVGA reduces memory and processing pressure, but detail and frame rate are lower than a compatible hardware-JPEG sensor at VGA. This is a hardware/processing tradeoff rather than webpage cropping [2], [6].

DHCP addresses may change after hotspot reconnection. Combining mDNS names, serially printed numeric endpoints, and UART camera announcements provides three discovery paths. The browser may report connection changes during refresh or Wi-Fi recovery; the firmware interprets those events conservatively and stops motion. This interface organization is consistent with mobile teleoperation work that combines wireless control and observation on the operator device [7].

Intermittent resets showed that successful USB commands did not prove vehicle-level power integrity. Motor current transients and switching noise can disturb the logic rail or common reference. Supplying motor voltage directly from the battery, deriving a regulated 5-V rail with the LM2596, and keeping grounds common produced the intended electrical structure. The L298N still has bipolar-transistor conduction loss, so battery condition, wiring resistance, module temperature, and stall current remain operating limits [3], [4].

V. LIMITATIONS AND EXTENSION POTENTIAL

The rover has no wheel encoders, current sensor, battery telemetry, or distance sensor. It therefore cannot report speed, estimate travelled distance, detect a stalled wheel, or automatically avoid an obstacle. Open-loop turning also depends on surface friction and motor matching. These limitations do not affect the completed manual-control objectives, but they define the measurements and hardware required for repeatable autonomy.

The camera fallback is suitable for short-range navigation and inspection, yet QVGA software-JPEG streaming limits fine detail and simultaneous viewers. A compatible hardware-JPEG camera would improve resolution and reduce conversion load [2]. Network performance could be characterized by recording command-to-actuation delay, frame rate, packet loss, useful range, and recovery time under repeatable conditions.

The verified platform can support line following with reflectance sensors, ultrasonic or time-of-flight obstacle detection, wheel encoders for closed-loop speed, inertial sensing, and battery monitoring. Additions should preserve current stop authority, documented pin ownership, separated power rails, and staged testing.

Face recognition is another possible extension because the ESP32-CAM already provides a network video source and the operator device shares the network. Prior mobile-robot work

has combined phone control with off-board visual processing [7]. A later implementation could send frames to a laptop and use embeddings derived from ArcFace [8]. This is not claimed as completed ESP32 work; consent, enrollment, distance, image quality, and false matches require a separate evaluation.

VI. Conclusion

Neural Rover completes the CSE499A objectives by integrating tested browser control, live pan-tilt video, documented power distribution, and fail-safe stopping in a four-wheel platform. Its verified two-controller architecture provides a stable base for later sensing, closed-loop motion, and laptop-assisted vision.

References

[1] Espressif Systems, "ESP32 series datasheet," ver. 5.2, 2025. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf

[2] Espressif Systems, "ESP32 camera driver," *esp32-camera*. [Online]. Available: https://github.com/espressif/esp32-camera

[3] STMicroelectronics, "L298 dual full-bridge driver," datasheet. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[4] Texas Instruments, "LM2596 SIMPLE SWITCHER power converter 150-kHz 3-A step-down voltage regulator," datasheet. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf

[5] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Dec. 2011, doi: 10.17487/RFC6455.

[6] J. A. Sahel, "CSE499 Neural Rover project source code and documentation," GitHub repository, 2026. [Online]. Available: https://github.com/Jubayed-Sahel/CSE499

[7] P. Papcun, I. Zolotova, and K. Tafsi, "Control and teleoperation of robot Khepera via Android mobile device through Bluetooth and WiFi," *IFAC-PapersOnLine*, vol. 49, no. 25, pp. 188–193, 2016, doi: 10.1016/j.ifacol.2016.12.032.

[8] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2019, pp. 4690–4699, doi: 10.1109/CVPR.2019.00482.